

Streams

¿Qué es un Stream?

Un Stream es una secuencia de elementos que pueden ser procesados con operaciones. Por ejemplo:

```
Integer[ ] myNums = { 1, 2, 3};  
Stream<Integer> myStream = Arrays.stream(myNums);  
System.out.println(myStream); // ¿Qué muestra?
```

Los Streams no son estructuras de datos para organizar los datos como un List o un array; en vez de eso podemos considerarlo como **un flujo de datos** a través del Stream. La fuente de los datos de myStream es myNums, una matriz de tres enteros, 1, 2, 3. Es por eso que cuando intentamos mostrar la transmisión, no vemos cualquier dato; sólo vemos una descripción críptica del objeto que describe cómo para llegar a los datos.

```
Integer[ ] myNums = {1, 2, 3};  
Stream<Integer> myStream = Arrays.stream(myNums);  
long numElements = myStream.count( ); // Toma el número de elementos del  
// Stream  
System.out.println("Número de elementos en el stream: "+ numElements);
```

Aquí, hemos agregado una operación count() a myStream. El método count() simplemente devuelve el recuento de elementos en el stream como un valor long. Al obtener un número desde el stream, el recuento de elementos, significa que el stream está acabado. Es decir, **no podemos realizar más operaciones en el stream** porque el stream ha sido convertido en un número por la operación count(). Decimos que la operación count() es una "**operación terminal**" porque el flujo termina allí: no fluyen más datos a través del stream después de que se realiza el count(). El verdadero poder de los streams proviene de las "operaciones intermedias" que se pueden realizar entre la fuente y el final de la secuencia.

Al especificar múltiples operaciones en una secuencia, básicamente estás definiendo un conjunto de cosas que quieres hacer en un orden particular a los datos en la fuente.

```
Integer[ ] myNums = {1, 2 ,3};  
Stream<Integer> myStream = Arrays.stream(myNums);  
long numElements = myStream.filter((i) -> i >1).count( );  
// Añade una operación intermedia para filtrar el stream.  
// Operación terminal, cuenta los elementos.  
System.out.println("Número de elementos > 1: " + numElements);
```

Observe que el método filter() de myStream produce un stream. Es un secuencia ligeramente modificada que consta sólo de elementos cuyos valores son mayores que

1. Ahora estamos llamando al método `count()` en esa secuencia filtrada, en lugar de al `myStream` original. Las operaciones intermedias siempre producen otro stream, así es como podemos encadenar múltiples operaciones juntas para manipular los datos a medida que fluyen en el stream. Mientras sigamos haciendo operaciones intermedias, mantenemos el stream de datos fluyendo; **sólo termina cuando realizamos una operación terminal como `count()`**.

```
Integer[ ] myNums = {1, 2 ,3};
Stream<Integer> myStream = Arrays.stream(myNums);

long numElements = myStream.filter((i) -> i >1).count( ); // Operación terminal
System.out.println("Número de elementos > 1: " + numElements);

numElements = myStream.filter((i) -> i > 2).count( );
System.out.println("Número de elementos > 2: " + numElements); // Excepción!!!
```

Esto produce una excepción!!! Una vez que realizamos una operación terminal el stream está acabado. En Java, los streams son objetos livianos, para que pueda crear múltiples streams si se necesita:

```
Integer[ ] myNums = {1, 2 ,3};
Stream<Integer> myStream = Arrays.stream(myNums);
long numElements = myStream.filter((i) -> i >1).count( );
System.out.println("Número de elementos > 1: " + numElements);

numElements = Arrays.stream(myNums) .filter((i) -> i > 2) .count( );
// filtrado por >2 en un stream Nuevo
System.out.println("Número de elementos >2: " + numElements);
```

En Java, un stream es un objeto que obtiene sus datos de una fuente, pero no almacenar ningún dato en sí mismo. Los datos que fluyen a través del stream se pueden operar, varias veces si queremos, con operaciones intermedias, como `filter()`. El stream finaliza cuando usamos una operación terminal, como `count()`, y una vez que hemos usado el stream, no podemos reutilizarlo.

Como crear un stream

Crear un Stream desde una colección

Vamos a crear un stream de una List:

```
List<Double> tempsEnMadrid = Arrays.asList(123.6, 108.0, 113.0, 112.5, 105.8, 117.0);
System.out.println("Número de días sobre 110: " + tempsEnMadrid
    .stream( )           // stream el List de Doubles
    .filter(t -> t > 110.0) // filtra el stream
    .count( ));          // cuenta los Doubles que pasan el filtro
```

El método **stream()** es heredado por todas las clases que implementan **Collection**. La signatura del método **stream()** es:

```
default Stream<E> stream( )
```

stream() devuelve un objeto de tipo **Stream**, con un parámetro genérico de tipo de objeto, lo que significa que el stream es un stream de cualquier tipo de objeto. Partiendo la línea del ejemplo anterior:

```
Stream<Double> tempStream = tempsEnMadrid.stream( );  
System.out.println("Número de días sobre 110: "  
+ tempStream.filter( t -> t > 110.0).count( ));
```

No olvidemos que un **Map** (**HashMap**, **TreeMap**, etc.) no es una colección heredando de **Collection**. Si desea transmitir un mapa, primero debemos usar el método **entrySet()** para convertir el **Map** en un **Set**, que es un tipo de **Collection**:

```
Map<String, Integer> myMap = new HashMap<String, Integer>( );  
myMap.put("Boi",6); myMap.put("Zooey",3); myMap.put("Charis",8);  
System.out.println("Número de items en el mapa con valor > 4: " + myMap  
.entrySet( )           // Toma un Set de objetos Map.Entry  
.stream( )             // stream del Set  
.filter(d -> d.getValue( ) > 4) // Filtra los objetos Map.Entry  
.count( ));           // Cuenta los objetos
```

Si intentamos transmitir el **Map** directamente, obtendremos un error en tiempo de compilación. En su lugar, primero llamamos a **entrySet()** en el **Map** y luego llamamos a **stream()** en el **Set** resultante. Luego llamamos a la operación **filter()** en este stream y obtenemos un objeto **Map.Entry** como argumento para la lambda Predicate que pasamos al filtro.

Construir un Stream con **Stream.of()**

La signatura de este método es:

```
static <T> Stream<T> of ( T... values)
```

Lo que significa que podemos proporcionar al método **of()** cualquier número de argumentos, y obtener una secuencia ordenada de esos valores.

```
Integer[ ] myNums = {1, 2 ,3};  
Stream<Integer> myStream = Stream.of(myNums);
```

Luego usamos **myStream** como lo hicimos antes para filtrar y contar los elementos.

Podemos hacer que ese código sea aún más corto omitiendo la declaración de la matriz en conjunto y suministrar valores enteros directamente, así:

```
Stream<Integer> myStream = Stream.of(1 ,2, 3);
```

Crear un Stream desde un Array

```
String[ ] dogs = { "Boi", "Zooey", "Charis" };  
Stream<String> dogStream = Arrays.stream(dogs);  
System.out.println("Número de elementos en el array: " + dogStream.count( ));
```

Otra forma es usando **Stream.of()**:

```
Stream<String> dogStream = Stream.of(dogs);
```

Crear un Stream desde un File

Podemos usar **Files** para leer datos de un archivo; El método estático **lines()** de Files devuelve un Stream, para que podamos transmitir los datos utilizando el archivo como fuente. Aquí está la signatura del método **Files.lines()**:

```
public static Stream<String> lines(Path path) throws IOException
```

Para crear un stream desde un archivo escribimos:

```
Stream<String> stream = Files.lines( Paths.get(filename) );
```

Y para procesar los datos del stream:

```
stream.forEach( line -> ...hacer algo con la línea de datos desde el archivo... )
```

El método **forEach()** en un stream funciona de manera muy similar al método **forEach()** en una colección. Toma un **Consumer**, que podemos representar con un expresión lambda, y procesa cada línea del archivo en el cuerpo de la lambda. El método **Files.lines()** proporciona **una línea a la vez** desde el archivo como elemento de datos en la secuencia, lo que hace que el procesamiento de datos del archivo fácil.

```
class DVDInfo {  
    String title; String genre; String leadActor;  
    DVDInfo(String t, String g, String a) {  
        title = t; genre = g; leadActor = a;  
    }  
    public String toString( ) {  
        return title + " / " + genre + " / " + leadActor;  
    }  
}
```

```

    // getters y setters aquí.
}
public class DVDs {
    public static void main(String[ ] args) {
        List<DVDInfo> dvds = loadDVDs("dvdinfo.txt");
        dvds.forEach(System.out::println);
    }

    public static List<DVDInfo> loadDVDs(String filename) {
        List<DVDInfo> dvds = new ArrayList<DVDInfo>( );
        try ( Stream<String> stream = Files.lines(Paths.get(filename))) {
            stream.forEach( line -> {
                String[ ] dvdItems = line.split("/");
                DVDInfofor dvd = new DVDInfo(dvdItems[0], dvdItems[1], dvdItems[2] );
                dvds.add(dvd);
            });
        }catch (IOException) {
            System.out.println("Error leyendo DVD");
            e.printStackTrace( );
        }
        return dvds;
    }
}

```

Al procesar datos de un archivo usando una secuencia con `forEach( )` como nosotros haga aquí, recuerde que dentro de la expresión lambda pasamos a `forEach( )`, **todas las variables deben ser finales o efectivamente finales**, por lo que no podemos modificar una variable directamente. Sin embargo, podemos agregar o modificar los campos de un objeto desde adentro la expresión lambda.

El método `forEach( )` de `Stream` es otro ejemplo de operación terminal (junto con `count( )`, que vimos anteriormente). No produce otro stream; más bien toma cada elemento que fluye por el stream y lo consume (con un `Consumer`). No produce nada (es decir, `forEach( )` es `void`).

Streams de valores primitivos

También hay streams de primitivos diseñados para evitar autoboxing, para doubles, ints y longs. Estos son `DoubleStream`, `IntStream` y `LongStream`, respectivamente. Entonces, para crear un `DoubleStream`:

```
DoubleStream s3 = DoubleStream.of(406.13, 406.42, 407.18, 409.01);
```

Hay que tener en cuenta la diferencia entre un **`Stream<Double>`** y **`DoubleStream`**. El primero es un stream de **objetos Double**; el segundo es un stream de **valores double**.

Resumen de métodos para crear Streams

Interface/Class	Creates a...	With method...
Collection	Stream<E>	stream()
Arrays	Stream<T>	stream(T[] array) stream(T[] array, int startInclusive, int endExclusive)
Arrays	IntStream, DoubleStream, LongStream	stream(int[] array), stream(double[] array), stream(long[] array) (and versions with start/end like Stream)
Files	Stream<String>	lines(Path path), lines(Path path, Charset cs)
Stream	Stream<T>	of(T... values), of(T t)
DoubleStream	DoubleStream	of(double... values), of(double t)
IntStream	IntStream	of(int... values), of(int t)
LongStream	LongStream	of(long... values), of(long t)

¿Por qué Streams?

La razón principal para usar streams es cuando comenzamos a hacer múltiples operaciones intermedias. Hasta ahora, hemos estado realizando sólo una operación intermedia: un filtro, usando una variedad de **Predicates** diferentes para filtrar los datos que obtuvimos del stream antes de contarlo. Sin embargo, cuando usamos múltiples operaciones intermedias, comenzamos a ver los beneficios de los streams.

```
List<String> names = Arrays.asList("Boi", "Charis", "Zooey", "Clover", "Aiko");
names.stream() // Crea el stream
.filter( s -> s.startsWith("B") || s.startsWith("C")) // Filtra por la primera letra
.filter(S -> s.length() > 3) // Filtra por longitud
.forEach(System.out::println);
```

La tubería Stream

Una tubería (pipeline) de flujo (stream) consta de tres partes: una **fuentes**, cero o más **operaciones intermedias**, y una **operación terminal**. La fuente describe de dónde provienen los datos; las operaciones intermedias operan en el stream y producen otro stream, quizás modificado; y la operación terminal finaliza la secuencia y normalmente produce un valor o alguna salida.

```
List<String> names = Arrays.asList("Boi", "Charis", "Zooey", "Clover", "Aiko");
names.stream( ) // La fuente
.filter( s -> s.startsWith("B") || s.startsWith("C")) // Operación intermedia
.filter(S -> s.length( ) >3) // Operación intermedia
.forEach(System.out::println); // Operación terminal
```

Bajo algunas circunstancias, podemos paralelizar streams para aprovechar la arquitectura subyacente del sistema y **hacer cálculos paralelos** muy fácilmente. No todas los streams pueden ser paralelizados, pero algunos pueden.

Los Streams son perezosos

En el ejemplo anterior los streams que van surgiendo a lo largo de la tubería en realidad no se materializan hasta que se ejecuta una operación terminal (en este caso `forEach( )`). Esta pereza hace que los streams sean más eficientes porque el JDK puede realizar optimizaciones para combinar las operaciones de manera eficiente, para operar en los datos en un solo pase, y para reducir las operaciones en los datos siempre que sea posible.

Operando en Streams

La expresión **"map->filter->reduce"** es una abstracción general para describir funciones que operan en una secuencia de elementos. Eso describe perfectamente lo que a menudo hacemos con los streams: podríamos "mapear" una entrada a una salida ligeramente diferente, luego "filtrar" esa salida según algunos criterios, y finalmente "reducir" a un solo valor o a una salida impresa. "map-filter-reduce", simplemente está especificando una secuencia de operaciones para ser realizadas en los datos en la fuente de la transmisión que conduce a un resultado. Con las transmisiones, tomamos la abstracción map-filter-reduce y la traducimos en operaciones utilizando los métodos **map()**, **filter()** y **reduce()** (y sus variaciones) en una secuencia.

Ya hemos visto el método **filter()** de Java Stream. Como dijimos, `filter( )` toma un Predicate y prueba cada elemento en el stream de la tubería, produciendo un stream de elementos que pasan la prueba.

El método **map()** de Stream es otra operación intermedia; sin embargo, a diferencia de `filter( )`, `map( )` no elimina los elementos de un stream; más bien, `map( )` transforma

elementos de un stream. Por ejemplo, `map()` podría calcular el cuadrado de un número, mapeando una stream de números a un stream de sus cuadrados o `map()` podría obtener la edad de una Persona para que el siguiente paso (por ejemplo, un filtro) puede encontrar edades mayores de 21 años. El método `map()` toma una Function. El propósito de una Function es transformar un valor. El método **`apply()`** de **Function** toma un valor y produce otro valor (no necesariamente del mismo tipo), por lo que la **Function** que pasamos a **`map()`** "mapeará" un valor a otro. El método funcional de Function `apply()` que pasamos al `map()` es llamado por `map()` detrás de escena, y el valor devuelto por `apply()` se pasa a la siguiente operación en la tubería de stream.

Reduce es tanto un método general, **`reduce()`**, como un método específico como `count()` y otros que veremos en breve. **Todas las reducciones son también operaciones terminales.** Las operaciones de reducción están diseñadas para combinar múltiples entradas en un resultado resumido, que podría ser un número único, como el que la operación de reducción `count()` produce, o una colección de valores.

El método de Stream, `reduce()`, es un método general para reducir un stream a un valor; la versión básica de `reduce()` toma una BiFunction (una Function cuyo método `apply()` toma dos argumentos y produce un valor) y lo aplica a pares de elementos del stream, devolviendo un valor. Podemos pensar en `reduce()` como una forma de "acumular" un resultado, como sumar los valores de un stream. Reducciones como `count()`, `sum()` y `average()` son definidas como métodos de streams; si queremos una función de reducción personalizada podemos definirla usando `reduce()`.

Definiremos ahora una List de Integers, tomaremos el cuadrado de cada Integer, probamos para ver si el cuadrado es mayor que 20, y si es así, agregaremos 1 a un conteo, result. Primero en la forma de **Java 7**:

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);
long result = 0;
for( Integer n : nums) {
    int square = n* n;
    if ( square > 20 ) {
        result++;
        System.out.println("Cuadrado de " + n + "es: " + square);
    }
}
System.out.println("Resultado: " + result);
```

Y ahora en el manera de **Java 8**:

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);
long result = nums.stream()
    .map( n -> n*n)
    .filter(n -> n > 20)
    .count()
    System.out.println("Resultado (stream): " + result);
```


Tengamos en cuenta que no podemos imprimir el número con su cuadrado usando el modo de los streams. ¿Por qué? Porque para cuando llegamos a la operación de reducción **count()**, los únicos elementos a los que tenemos acceso en el stream son los cuadrados de los números. Los números originales se almacenan solo en la **List** original. Una solución:

```
long result = nums.stream( )
    .peek(n -> System.out.print("El número es: " + n + " , "))
    .map(n -> n*n)
    .filter(n -> n > 20)
    .peek(n -> System.out.print("El cuadrado es : " + n + " , "))
    .count( );
```

El método **peek()** es una operación intermedia que le permite "mirar" en el stream a medida que pasan los elementos. Toma un **Consumer** y produce exactamente el mismo stream que es llamado, por lo que no cambia los valores ni los filtra de manera alguna.

Map-Filter-Reduce con **average()** y **Optionals**

Imaginemos que tenemos un equipo que está leyendo una vez por semana, y desea encontrar el promedio de las lecturas que ha medido este año. Sabemos que las lecturas probablemente deberían estar entre 406 y 407, por lo que decidimos ignorar las lecturas que son inferiores a 406 o mayor que 407 porque pueden ser errores:

```
class Reading {
    int year;
    int month;
    int day;
    double value;
    Reading(int year, int month, int day, double value) {
        this.year = year; this.month = month ; this.day = day; this.value = value;
    }
}
```

Cada lectura se almacena con su valor, junto con el año, mes y día, en un objeto **Reading**, y los objetos **Reading** se almacenan en una **List**:

```
List<Reading> readings = Array.asList(
    new Reading(2017, 1, 1, 405.91) ,
    new Reading(2017, 1, 8, 405.98) ,
    new Reading(2017, 1, 15, 406.14) ,
    new Reading(2017, 1, 22, 406.48) ,
    new Reading(2017, 1, 29, 406.20) ,
    new Reading(2017, 2, 5, 407.12) ,
    new Reading(2017, 2, 12, 406.03) );
```

Sólo estamos interesados en una porción de los valores de la lectura, ya que buscamos obtener la media. Podemos asignar cada lectura a su valor, esencialmente convirtiendo la lectura a un sólo valor double. Pero no podemos usar `map( )` porque `map( )` toma una Function cuyo método `apply( )` **toma un objeto y devuelve un objeto**.

```
<R> Stream<R> map ( Function<? super T, ? extends R> mapper)
```

`map( )` toma una función (llamada `mapper` en la firma de encima). `map( )` aplica esa Function (llamando al método `apply( )`) en cada valor del stream. Recordemos que el método `apply( )` de Function toma un objeto y devuelve un objeto. Entonces, si llamamos a `map( )` en un Stream de objetos, como queremos hacer aquí con nuestro stream de objetos `Reading`, lo que recuperamos es también una corriente de objetos. Pero `Reading.value` es un double, no un Double, así que lo que realmente queremos recuperar es un stream de doubles.

¿Hay alguna solución? ¡Sí, por supuesto! El método **`Stream.mapToDouble( )`** toma un **`ToDoubleFunction`**, cuyo método **`applyAsDouble( )`** toma un objeto y devuelve un double:

```
double applyAsDouble(T value)
```

Con esto:

```
readings.stream( ).mapToDouble(r -> r.value)
```

Lo que obtenemos de vuelta del método **`mapToDouble( )`** del Stream que creamos con **`readings.stream( )`** es un **`DoubleStream`**:

```
DoubleStream mapToDouble(ToDoubleFunction<? super T > mapper))
```

Entonces, al llamar a **`mapToDouble( )`**, hemos convertido el flujo inicial de lectura objetos (**`Stream <Reading>`**) en una secuencia de doubles, y esa secuencia tiene el tipo **`DoubleStream`**.

```
readings.stream( ).mapToDouble(r -> r.value)
```

Hemos mapeado el flujo de objetos `Reading` en un stream de doubles, y a continuación, queremos filtrar el stream para mantener solo los valores mayores o igual a 406.0 y menor a 407.0:

```
readings.stream( )  
    .mapToDouble(r -> r.value)  
    .filter (v -> v >= 406.0 && v < 407.00)
```

La operación de reducción que vamos a usar es el método **`average( )`**, que no está en el interfaz `Stream`. Eso es porque `average( )` es un método de la interfaz **`DoubleStream`** (y

su primos primitivos, **IntStream** y **LongStream**). Eso tiene sentido porque **average()** está diseñado para trabajar con números, no con objetos.

```
OptionalDouble average( )
```

OpcionalDouble es uno de los varios tipos de opcionales que son nuevos en Java 8, que representan valores que pueden o no estar allí. Pensemos en ello. ¿Qué pasaría si la **List** de **Readings** estuviese vacía? Los filtros funcionarían sin problema, pero cuando tratásemos de calcular el promedio, estaríamos dividiendo entre cero (mal asunto). Tendríamos de alguna forma que chequear que hay algún valor en la colección para no tener ese problema. Los **Optionals** nos ayudan a prevenir ese problema. No tiene sentido devolver null en esta situación porque estamos esperando un double primitivo. Y no tiene sentido devolver 0.0 porque ese podría ser un resultado válido para el promedio. En cambio, **devolvemos un OpcionalDouble, que es un valor double que podría o no estar allí.**

```
OptionalDouble avg = readings.stream( )  
    .mapToDouble(r -> r.value)  
    .filter( v -> v >= 406.0 && 407.00 )  
    .average( );
```

Si tratamos de imprimir el valor del promedio:

```
System.out.println("Promedio de las lecturas: " + avg);
```

Se genera algo envuelto en un OptionalDouble, como:

```
Promedio de las lecturas: OptionalDouble[406.2125]
```

Podríamos evitar esto usando:

```
System.out.println("Promedio de las lecturas: " + avg.getAsDouble( ));
```

```
Promedio de las lecturas: 406.2125
```

Sin embargo, el problema con este código es si el **OpcionalDouble** está vacío, entonces obtendrá una **NoSuchElementException**. Una mejor manera de escribir este código es verificar primero para asegurarse de que haya un valor presente en el **OpcionalDouble** con el método **isPresent()**:

```
if(avg.isPresent) {  
    System.out.println(" Promedio de las lecturas: " + avg.getAsDouble( ));  
} else {  
    System.put.println("optional vacio");  
}
```

Reduce

Hasta ahora has visto un par de reducciones: `count()` y `average()`. El método `count()` se define tanto para `Stream` como para las versiones primitivas de `Stream`: `IntStream`, `LongStream` y `DoubleStream`, mientras que `average()` es definido solo para los stream de primitivos. Otros métodos útiles definidos en los flujos primitivos incluyen **`min()`**, **`max()`** y **`sum()`**, que dan como resultado el valor mínimo en un stream, el valor máximo en un stream y la suma de los valores en un stream. Al igual que `average()`, `min()` y `max()` también devuelven valores `Optional`.

```
OptionalDouble max = readings.stream()
    .mapToDouble(r -> r.value)
    .max();
if(max.isPresent()) {
    System.out.println("Máximo de todas las lecturas: " + max.getAsDouble());
} else {
    System.out.println("Optional vacío.");
}
```

Al igual que con **`average()`**, no tiene sentido tomar **`max()`** o **`min()`** de un corriente vacía, por ello un `Optional`. Sin embargo `DoubleStream.sum()` no, ya que se asume que el resultado de la suma de un stream vacío es 0.0.

```
List<Reading> readings2 = Arrays.asList(); // Lista vacía
double sum = readings2.stream().mapDouble(r -> r.value).sum();
System.out.println("Suma de todas las lecturas: " + sum);
```

Usando reduce()

Los métodos **`count()`**, **`average()`**, **`min()`**, **`max()`** y **`sum()`** son todos métodos de reducción ya definidos en los streams. También podemos definir nuestras propias reducciones utilizando el método **`reduce()`**. Reescribiremos el código para sumar el valores en la lista de readings (original, no vacía) usando **`reduce()`** en lugar de **`sum()`**. La signatura de **`DoubleStream.reduce()`** es:

```
OptionalDouble reduce( DoubleBinaryOperator op)
```

Los **BinaryOperators** son como **UnaryOperators** excepto sus funciones. Los métodos toman dos valores como argumentos, en lugar de uno. Los operadores son un caso especial de `Function` en el que los argumentos y el valor de retorno del método funcional deben ser del mismo tipo. Un `DoubleBinaryOperator` es un operador cuyo método funcional, `applyAsDouble()`, toma dos valores `double` y devuelve un `double`.

Para escribir nuestro propio método para sumar todos los valores en el stream usando `reduce()`, pasamos un `DoubleBinaryOperator` a `reduce()`, que agrega el dos argumentos juntos y devuelve la suma:

```
OptionalDouble sum = readings.stream( )
    .mapToDouble(r -> r.value)
    .reduce((v1,v2) -> v1 +v2);
if(sum.isPresent()){
    System.out.println("La suma de todas las lecturas es: " + sum.getAsDouble( ));
}
```

El método **`reduce()`** suma todos los valores en un stream, dos a la vez, para obtener la suma total. En otras palabras, calcula el resultado final al "acumular" los valores que provienen del stream.

Echemos otro vistazo al código anterior y observemos que cuando cambiamos de **`sum()`** a **`reduce()`**, tuvimos que volver a usar **`OpcionalDouble`** para el resultado. Eso es porque **`reduce()`** es una función de reducción general que toma cualquier **`DoubleBinaryOperator`**.

Sabemos que el método **`applyAsDouble()`** debe producir un número, pero en la situación donde el stream está vacío, el método **`applyAsDouble()`** nunca será llamado, así que necesitamos una manera de decir que puede que no haya resultado en absoluto. Hay otro método **`reduce()`** en **`DoubleStream`** que toma una identidad junto con una función de acumulador y devuelve un doble:

```
double reduce( double identity, DoubleBinaryOperatos op)
```

El argumento de identidad cumple dos roles: proporciona un valor inicial y un valor a devolver si el stream está vacío. Porque tenemos un valor para devolver cuando el stream está vacío, ya no tenemos que usar un **`OpcionalDouble`** porque siempre recuperaremos un valor:

```
double sum = readings.stream( )
    .mapToDouble(r -> r.value)
    .reduce( 0.0, (v1,v2) -> v1 +v2);
System.out.println( "La suma de todas las lecturas es: " + sum );
```

Métodos mapea-filtra-reduce

Interface	Method	Returns...
Stream	<code>map(Function<? super T, ? extends R> mapper)</code>	<code>Stream<R></code>
Stream	<code>mapToDouble(ToDoubleFunction<? Super T> mapper),</code> <code>mapToInt(ToIntFunction<? Super T> mapper),</code> <code>mapToLong(ToLongFunction<? Super T> mapper)</code>	<code>DoubleStream,</code> <code>IntStream,</code> <code>LongStream</code>
DoubleStream	<code>map(DoubleUnaryOperator mapper),</code> <code>mapToInt(DoubleToIntFunction mapper),</code> <code>mapToLong(DoubleToLongFunction mapper),</code> <code>mapToObj(DoubleFunction<? Extends U> mapper)</code>	<code>DoubleStream,</code> <code>IntStream,</code> <code>LongStream,</code> <code>Stream<U></code>

Interface	Method	Returns...
IntStream	Like DoubleStream, but for ints	
LongStream	Like DoubleStream, but for longs	
Stream	filter(Predicate<? super T> predicate)	Stream<T>
DoubleStream	filter(DoublePredicate predicate)	DoubleStream
IntStream	filter(IntPredicate predicate)	IntStream
LongStream	filter(LongPredicate predicate)	LongStream
Stream	reduce(BinaryOperator<T> accumulator), reduce(T identity, BinaryOperator<T> Accumulator)	Optional<T>, T
DoubleStream	reduce(DoubleBinaryOperator op), reduce(double identity, DoubleBinaryOperator op)	OptionalDouble, double
IntStream	reduce(IntBinaryOperator op), reduce(int identity, IntBinaryOperator op)	OptionalInt, int
LongStream	reduce(LongBinaryOperator op), reduce(long identity, LongBinaryOperator op)	OptionalLong, long
Stream	count()	long
DoubleStream, IntStream, LongStream	count(), sum()	long, double, int, long
DoubleStream, IntStream, LongStream	average()	OptionalDouble
DoubleStream, IntStream, LongStream	max(), min()	OptionalDouble, OptionalInt, OptionalLong

Acumulaciones asociativas

Probemos a hacer algo parecido para construir nuestra propia función para calcular el promedio:

```
OptionalDouble avgWithReduce = readings.stream()
    .mapToDouble(r -> r.value)
    .filter(v -> v >= 406.0 && v < 407.00)
    .reduce((v1,v2) -> (v1 + v2) / 2);
if(avgWithReduce.isPresent()) {
    System.out.println("Promedio de las lecturas" + avgWithReduce.getAsDouble());
} else {
    System.out.println("Optional vacío.");
}
```

Al ejecutarlo, podemos comprobar que la respuesta es diferente de la obtenida con anterioridad. Fijémonos en la que hemos puesto:

```
reduce((v1,v2) -> (v1 + v2) / 2);
```

Como hicimos con sum, estamos pasando un **DoubleBinaryOperator** para reducir, cuyo método funcional toma dos valores. Estamos sumando esos dos valores y dividiendo por dos. La idea es que **reduce()** opera en dos valores del stream a la vez, por lo que obtendremos un promedio de dos valores y luego promediamos ese resultado con el siguiente valor que viene del stream. El problema es que **average()**, a diferencia de **sum()**, **max()** y **min()**, **no es asociativo**. Un operado asociativo cumple:

$$(A \text{ operador } B) \text{ operador } C = A \text{ operador } (B \text{ operador } C)$$

Resulta que **average()** **No** es un operador asociativo. Los operadores de reducción deben ser asociativos para funcionar correctamente con streams. Es por eso que no podemos definir nuestra propia función promedio con **reduce()**; **El promedio no es asociativo**. Entonces, para tomar el promedio de un stream, deberemos usar el método **average()**.

Optionals

Tabla con métodos **Stream** que devuelven **Optionals**:

Interface	Method	Returns...
Stream	findAny()	Optional<T>
Stream	findFirst()	Optional<T>
Stream	max(Comparator<? super T> comparator)	Optional<T>
Stream	min(Comparator<? super T> comparator)	Optional<T>
Stream	reduce(BinaryOperator<T> accumulator)	Optional<T>
DoubleStream	average()	OptionalDouble
DoubleStream	findAny(), findFirst()	OptionalDouble
DoubleStream	max(), min()	OptionalDouble
DoubleStream	reduce(DoubleBinaryOperator op)	OptionalDouble
IntStream, LongStream	Similar methods to DoubleStream	OptionalInt, OptionalLong

¿Qué es un Optional? Es un contenedor que puede o no contener un valor. Puede pensar en un opcional como un contenedor alrededor de un valor y ese contenedor proporciona varios métodos para determinar si hay un valor y, si es así, para obtener ese valor.

Los Optionals se muestran mucho con los streams porque, como hemos visto, si un stream está vacío en cualquier parte de la tubería, entonces existe la posibilidad de que no sea devuelto ningún valor por la operación terminal de una tubería de stream. Los opcionales proporcionan una forma para que el JDK maneje esa situación sin tener que lanzar una excepción en tiempo de ejecución. Los tipos de opcionales incluyen **Optional<T>** (para objetos) y tres opcionales primitivas, **OptionalDouble**, **OptionalInt** y **OptionalLong**.

```
Stream<Double> doubleStream = Stream.of(1.0, 2.0, 3.0, 4.0);
Optional<Double> aNum = doubleStream.findFirst( );
if (aNum.isPresent( )) {
    System.out.println("Primer número del stream de doubles: "+ aNum.get( ));
} else {
    System.out.println("El stream de doubles está vacío.");
}
```

Hay que darse cuenta de los doubles se están encuadrando automáticamente (autoboxing) en Doubles para crear el **Stream <Double>**. También hay que tener en cuenta que **OptionalDouble** no funciona aquí. ¿Por qué? Debido a que **findFirst()** está operando en un stream de Doubles, no en un stream de doubles.

Ahora que tenemos un valor opcional, primero debemos probar para ver si está presente, con **isPresent()**, antes de intentar obtener ese valor Double del Optional. Porque estamos trabajando con un objeto Double en lugar de un tipo primitivo, el método que utilizamos para obtener el valor es **get()**.

Remarcar que el método para **OptionalDouble** es **getAsDouble()**, también lo patrones para los nombres de los métodos para para opcionales y opcionales de primitivas:

- Obtener un valor desde un Optional<T>: **get()**
- Obtener un valor desde un OptionalDouble: **getAsDouble()**
- Obtener un valor desde un OptionalInt: **getAsInt()**
- Obtener un valor desde un OptionalLong: **getAsLong()**

Otra forma de probar para ver si un opcional contiene un valor y, si es así, obtener ese valor, es el método **ifPresent()**. Este método toma un **Consumer** y prueba para ver si el valor opcional está presente; si es así, entonces el valor sin envolver se pasa al **Consumer**:

```
Stream<Double> doubleStream = Stream.of(1.0, 2.0, 3.0, 4.0);
Optional<Double> aNum = doubleStream.findFirst( );
aNum.ifPresent(n -> System.out.println("Primer número del stream de doubles: " + n));
```

Hay que tener en cuenta que el tipo de n es **Double**, no **Opcional <Double>**, por lo que para imprimir n, simplemente escribimos n, no **n.get()**. Si el **doubleStream** está vacío, no verá ningún resultado en absoluto. También podemos crear nuestros propios opcionales:

```
Dog boi = new Dog("boi",30,6);
Optional<Dog> optionalBoi = Optional.of(boi);
optionalBoi.ifPresent(System.out::println);
```

¿Qué ocurre si boi es null?

```
boi = null;
Optional<Dog> optionalBoi = Optional.of(boi);
optionalBoi.ifPresent(System.out::println);
```

Se produce una excepción: **java.lang.NullPointerException**. Si nos encontramos en una situación en la que podría estar creando un optional de un objeto null, entonces debemos tomar una precaución adicional y usar el método **opcional.ofNullable()** en lugar de **Opcional.of ()**. El método **ofNullable()** crea el opcional si el objeto que pasa no es null; de lo contrario, crea un opcional vacío:

```
boi = null;
Optional<Dog> optionalBoi = Optional.ofNullable(boi);
optionalBoi.ifPresent(System.out::println);
if (!optionalBoi.isPresent( )) System.out.println("Boi debe ser nulo");
```

También podemos crear optionals vacíos directamente. Supongamos que tenemos una List de Objetos Dog. Podemos crear un **Optional<Dog>** vacío y luego asignarle un valor más tarde transmitiendo la List de Dogs, usando **findFirst()** para encontrar el primer Dog:

```
Optional<Dog> emptyDog = Optional.empty( );
if (!emptyDog.isPresent( )) {
    System.out.println("EmptyDog debe estar vacío. ");
}
emptyDog = dogs.stream( ).findFirst( );
emptyDog.ifPresent(d -> System.out.println("Empty dog ya no está vacío."));
```

Otra forma de manejar un optional vacío es el método **orElse()**. Con este método obtenemos el valor en el optional, o, si ese optional está vacío, obtenemos el valor que especifiquemos con el método **orElse()**:

```
Optional<Dog> emptyDog = Optional.empty( );
Dog aDog = emptyDog.orElse(new Dog("Dog por defecto", 50, 10));
System.out.println("Un Dog: " + aDog);
```

Hay que probar también:

```
Dog boi = new Dog("boi", 30, 6);
Optional<Dog> optionalBoi = Optional.of(boi);
Dog a Dog = optionalBoi.orElse(new Dog("Dog por defecto", 50, 10));
System.out.println("Un Dog: " + a Dog);
```

Class	Method	Returns...
Optional	empty()	Optional<T>
Optional	get()	T
Optional	ifPresent(Consumer<? super T> consumer)	void
Optional	isPresent()	boolean
Optional	of()	Optional<T>
Optional	ofNullable(T value)	Optional<T>
Optional	orElse(T other)	T
OptionalDouble, OptionalInt, OptionalLong	Each has similar methods to the above methods that return primitive optionals or primitives for each type	double, int, long

Buscando y clasificando con Streams

El interfaz Stream proporciona varios métodos para buscar elementos en streams: **allMatch**, **anyMatch**, **noneMatch**, **findFirst** y **findAny**. Todas estas operaciones son operaciones terminales; es decir, todas devuelven un solo valor, no un stream.

Buscando para ver si un elemento existe

Los métodos **allMatch()**, **anyMatch()** y **noneMatch()** toman todos un **Predicate** para hacer una prueba de coincidencia y devolver un valor booleano.

```
class Dog {
    private String name;
    private int age;
    private int weight;
    public Dog(String name, int weight, int age) {
        this.name = name; this.weight = weight; this.age = age;
    }
    // getters y setters
    public String toString() {
        return this.name + " tiene " + this.age + " años y pesa " + this.weight + " kilos.";
    }
}
```

```

ArrayList<Dog> dogs = new ArrayList<>( );
Dog boi = new Dog("boi", 30, 6);
Dog clover = new Dog("clover", 35, 12);
Dog aiko = new Dog("aiko", 50, 10);
Dog zooey = new Dog("zooey", 45, 8);
Dog charis = new Dog("charis", 120, 78);
dogs.add(boi); dogs.add(clover); dogs.add(aiko);
dogs.add(zooey); dogs.add(charis);

```

Primero, buscaremos dogs cuyo peso sea mayor a 50 kilos y cuyos nombres comienzan con "c" usando **anyMatch()**. El método **anyMatch()** deja de buscar en cuanto encuentra al menos un dog cuyo nombre comience por "C":

```

boolean cNames = dogs.stream( )
    .filter(d -> d.getWeight( ) > 50)
    .anyMatch(d -> d.getName( ).startsWith("C"));
System.out.println("¿Hay algún perro de más de 50 Kilos y cuyo nombre empiece por 'C' " + cNames);

```

Ahora usemos **allMatch()** para encontrar si todos los dogs en la lista tienen una edad mayor de 5. Aquí, mapeamos el stream de dogs a un stream de sus edades primero y luego usamos **allMatch()** para verificar que cada dog tenga edad mayor que 5. **allMatch()** se detendrá y devolverá falso tan pronto como encuentre cualquier dog que falle esta prueba.

```

boolean isOlder = dogs.stream( )
    .mapToInt(d -> d.getAge( )) // Mapeamos de Dog a la edad del Dog (integer)
    .allMatch(a -> a>5);
System.out.println("¿Son todos los dogs mayores de 5 años? " + isOlder);

```

Finalmente, utilizaremos **noneMatch()** para asegurarnos de que ninguno de los dog en el stream se llama "rojo". Primero, mapeamos el flujo de dogs a un stream de nombres de dogs, y luego usamos **noneMatch()** para verificar el nombre:

```

boolean notRed = dogs.stream( )
    .map(d -> d.getName( ))
    .noneMatch(n -> n.equals("red"));
System.out.println("Ninguno de los perros se llama Red: " + notRed);

```

Buscando para encontrar y retornar un objeto

Todos estos métodos coincidentes determinan si existe una coincidencia o no. Si sí queremos recuperar un resultado de una partida, entonces podemos usar **findFirst()** o **findAny()**. Ninguno de estos métodos tiene un argumento, entonces primero debemos filtrar para reducir los elementos que podríamos estar buscando. Debido a que **filter()** podría filtrar todos los elementos del stream, dejándonos con un stream

vacío, podemos adivinar que **findFirst()** y **findAny()** ambos devuelven opcionales en caso de que sean llamados para streams vacíos, en cuyo caso, no hay un resultado válido y el opcional estará vacío.

```
Optional<Dog> c50 = dog.stream( )
    .filter(d -> d.getWeight( ) > 50)
    .filter(d -> d.getName( ).startsWith("C"))
    .findAny( );
c50.ifPresent(System.out::println);
```

¿Qué ocurre si la búsqueda encuentra más de una coincidencia? **findAny()** podría devolver cualquiera de los dogs en la secuencia (solo uno). No hay garantía de que será el primero.

No olvidemos que todos estos métodos de búsqueda y comparación son de **cortocircuito**. Eso significa que tan pronto como **findAny()** encuentre un dog que coincida, todo se detiene.

```
Optional<Dog> d5 = dogs.stream( )
    .filter(d -> d.getAge( ) > 5).peek(System.out::println)
    .findAny( );
d5.ifPresent(System.out::println);
```

Recordemos que **peek()** permite "mirar" en el stream. Toma un **Consumer** y devuelve exactamente el mismo stream desde el que se llama, por lo que no hace ningún cambio. Es simplemente una ventana a lo que está fluyendo por en el stream.

Clasificando

El interfaz **Stream** incluye un método **sorted()** que puede usar para ordenar un stream de elementos por orden natural o proporcionando un **Comparator** para determinar el orden.

```
Stream.of("Jerry", "George", "Kramer", "Elaine")
    .sorted( )
    .forEach(System.out::println);
```

Vamos a ordenar objetos más complejos:

```
class Duck implements Comparable<Duck> {
    String name;
    String color;
    int age;
    public Duck(String name, String color, int age) {
        this.name = name; this.color = color; this.age = age;
    }
}
```

```
// getters y setters aquí
public String toString() {
    return ( getName() + " es " + getColor() + " y tiene " + getAge() + " años.");
}
@Override
public int compareTo( Duck duck) {
    return this.getName().compareTo(duck.getName());
}
}
```

Usamos la List:

```
List<Duck> ducks = Array.asList(
    new Duck("Jerry", "yellow",3),
    new Duck("George", "brown",4),
    new Duck("Kramer", "mottled",6),
    new Duck("Elaine", "white",2) );

ducks.stream()
    .sorted()
    .forEach(System.out::println);
```

¿Qué pasa si queremos cambiar el criterio cuándo transmitimos los ducks? Podemos usar **sorted()** con un **Comparator**:

```
ducks.stream()
    .sorted((d1,d2) -> d1.getAge() - d2.getAge())
    .forEach(System.out::println);
```

Aquí, estamos proporcionando un **Comparator** para **sorted()** que se usará para ordenar (en lugar del método **compareTo()** en la clase Duck).

Arriba, pasamos el **Comparator** directamente al método **sorted()** usando una expresión lambda. Si quisiéramos, podríamos escribir el **Comparator** por separado, asignárselo a una variable y luego pasarle esa variable a **sorted()**.

```
Comparator<Duck> byAgeLambda = (d1,d2) -> d1.getAge() - d2.getAge();
ducks.stream()
    .sorted(ByAgeLambda)
    .forEach(System.out::println);
```

Comparator también tiene algunos métodos estáticos útiles que nos gustará conocer al definir comparadores para usar con streams: **comparing()**, **reverse()**, y **thenComparing()**.

Escribamos tres comparadores para clasificar los ducks por su color, por su nombre, y por su edad, utilizando el método **Comparator.comparing()**. El método **comparing()** toma una **Function** cuyo método funcional espera un propiedad para ordenar, como `Duck.age`, como argumento y devuelve un **Comparator** que compara objetos por esa propiedad.

```
Comparator<Duck> byColor = Comparator.comparing(Duck::getColor);
Comparator<Duck> byName = Comparator.comparing(Duck::getName);
Comparator<Duck> byAge = Comparator.comparing(Duck::getAge);
```

Ahora que tenemos estos comparadores, podemos usarlos para clasificarducks en varias combinaciones. Po edad:

```
ducks.stream( ).sorted(byAge).forEach(System.out::println);
```

o por edad de forma inversa:

```
ducks.stream( ).sorted(byAge.reversed( )).forEach(System.out::println);
```

También podemos ordenar por una propiedad y luego por otra propiedad, usando el método **thenComparing()**. **thenComparing()** devuelve un **Comparator** que compara primero por el **Comparator** al que lo llama y luego por el **Comparator** que le pasamos. Para ordenar por nombre, y luego por edad, escribiremos:

```
byName.thenComparing(byAge)
```

Podemos probarlo añadiendo algunos nuevos objetos a la lista:

```
List<Duck> ducks = Array.asList(
    new Duck("Jerry", "yellow",3),
    new Duck("George", "brown",4),
    new Duck("Kramer", "mottled",6),
    new Duck("Elaine", "white",2) );
new Duck("Jerry", "mottled",10),
new Duck("George", "white",12),
new Duck("Kramer", "brown",11),
new Duck("Elaine", "brown",13) );

ducks.stream( )
    .sorted(byName.thenComparing(byAge))
    .forEach(System.out::println);
```

O una variedad de otras comparaciones. Finalmente, otro método de Stream que es útil al ordenar streams es **distinct()**. Este método devuelve un stream con elementos distintos, así que si un elemento se repite en el stream, terminaremos con solo uno de ellos.

```
List<Duck> ducks = Array.asList(
    new Duck("Jerry", "yellow",3),
    new Duck("George", "brown",4),
    new Duck("Kramer", "mottled",6),
    new Duck("Elaine", "white",2) );
    new Duck("Huey", "mottled",2),
    new Duck("Louie", "white",4),
    new Duck("Dewey", "brown",6) );

ducks.stream( )
    .map(d -> d.getColor( ))
    .distinct( )
    .forEach(System.out::println);
```

Hemos deslizado algunas referencias de métodos en esta sección: **Duck :: getColor**, **Duck :: getName**, **Duck :: getAge** y **System.out :: println**. Recordemos que **System.out :: println** es una referencia de método: una abreviatura para una expresión lambda. Por ejemplo, reemplazamos una lambda expresión como esta:

```
Stream.of("Elain","Jerry","George","Kramer").forEach(n -> System.out.println(n));
```

Con una referencia a método como esta:

```
Stream.of("Elain","Jerry","George","Kramer").forEach(System.out::println);
```

Las referencias a métodos simplemente reemplazan las expresiones lambda que no hacen nada excepto llamar a otra función. Para esta referencia a método de instancia, System.out es la clase y println es el método de instancia. Usamos la sintaxis "::" para indicar la referencia al método.

Echemos otro vistazo a **Duck :: getColor**, **Duck :: getName** y **Duck :: getAge**. Estas también son referencias de método a métodos de instancia. Son un tipo de referencia a método que se refiere a un método no estático de una instancia de un tipo. Si escribiéramos esas referencias a método como expresiones lambda, se verían así:

```
Comparator<Duck> byColor = Comparator.comparing(d -> d.getColor( ));
Comparator<Duck> byName = Comparator.comparing(d -> d.getName( ));
Comparator<Duck> byAge = Comparator.comparing(d -> d.getAge( ));
```

Entonces, por ejemplo, el lambda **d -> d.getColor()** es una función cuyo método funcional toma un objeto de tipo Duck y devuelve un objeto de tipo String, que asigna un Duck a su color.

A continuación unas tablas con los métodos para buscar y clasificar Streams:

Interface	Method	Description
Stream	<code>allMatch(Predicate<? super T> predicate)</code>	Returns a boolean; true if all elements in the stream pass the Predicate test. Stops when any element fails the test.
Stream	<code>anyMatch(Predicate<? super T> predicate)</code>	Returns a boolean; true if any of the elements in the stream pass the Predicate test. Stops when any element passes the test.
Stream	<code>noneMatch(Predicate<? super T> predicate)</code>	Returns a boolean; true if none of the elements in the stream pass the Predicate test. Stops when any element passes the test.
Stream	<code>findFirst()</code>	Returns an Optional<T> with the first element from the stream or an empty optional if the stream is empty.
Stream	<code>findAny()</code>	Returns an Optional<T> with an element from the stream (no guarantees about which one!) or an empty optional if the stream is empty.
DoubleStream, IntStream, LongStream	<code>allMatch(DoublePredicate predicate),</code> <code>allMatch(IntPredicate predicate),</code> <code>allMatch(LongPredicate predicate)</code>	Returns a boolean; true if all elements in the stream pass the Predicate test. Stops when any element fails the test.
DoubleStream, IntStream, LongStream	<code>anyMatch(DoublePredicate predicate),</code> <code>anyMatch(IntPredicate predicate),</code> <code>anyMatch(LongPredicate predicate)</code>	Returns a boolean; true if any of the elements in the stream pass the Predicate test. Stops when any element passes the test.

Interface	Method	Description
DoubleStream, IntStream, LongStream	noneMatch(DoublePredicate predicate) , noneMatch(IntPredicate predicate) , noneMatch(LongPredicate predicate)	Returns a boolean; true if none of the elements in the stream pass the Predicate test. Stops when any element passes the test.
DoubleStream	findFirst()	Returns an OptionalDouble: the first element in the stream or an empty optional if the stream is empty.
IntStream	findFirst()	Returns an OptionalInt: the first element in the stream or an empty optional if the stream is empty.
LongStream	findFirst()	Returns an OptionalLong: the first element in the stream or an empty optional if the stream is empty.
DoubleStream	findAny()	Returns an OptionalDouble: an element in the stream or an empty optional if the stream is empty.
IntStream	findAny()	Returns an OptionalInt: an element in the stream or an empty optional if the stream is empty.
LongStream	findAny()	Returns an OptionalDouble: an element in the stream or an empty optional if the stream is empty.
Stream, DoubleStream, IntStream, LongStream	sorted()	Returns a Stream<T> (or primitive stream type). Sort elements of a stream by natural order
Stream	sorted(Comparator<? super T> comparator)	Returns a Stream<T>. Sort elements of a stream by the provided Comparator.
Stream, DoubleStream, IntStream, LongStream	distinct()	Returns a Stream<T> (or primitive stream type) with distinct elements. For Stream<T>, the Object .equals (Object) test is used.

No modifique la fuente del Stream

Aunque Java está lejos de ser un lenguaje de programación funcional, los streams son un paso hacia un estilo de programación funcional que fomenta la idea de evitar cambios de estado en la estructura de datos original que siendo procesado a través de una stream.

En este punto, vamos a compartir algo realmente importante sobre los streams. Es decir, nunca debemos intentar modificar la fuente de un stream desde dentro de la tubería del stream. Java no dará un error en tiempo de compilación si lo intentamos y puede que ni siquiera de un error en tiempo de ejecución, aunque **los resultados no estarán garantizados**.

```
dogs.stream( ).filter(d -> {
    if (d.getWeight( ) < 50) {
        dogs.remove(d);
        return false;
    }
    return true;
}).forEach(System.out::println);
```

Podemos pensar que veremos una lista de los dogs cuyo peso es mayor o igual a 50, y podríamos pensar que la lista de perros ahora solo contendrá perros cuyo peso es mayor o igual a 50, pero lo que probablemente obtener en su lugar es una excepción. La mejor manera de hacerlo es recolectar los dogs al final de la tubería del stream hacia una nueva colección. Y, por supuesto, Java proporciona una manera fácil de hacer exactamente eso con el método **Stream.collect()**.

```
ArrayList<Dog> dogs = new ArrayList<>( );
Dog boi = new Dog("boi", 30, 6);
Dog clover = new Dog("clover", 35, 12);
Dog aiko = new Dog("aiko", 50, 10);
Dog zooey = new Dog("zooey", 45, 8);
Dog charis = new Dog("charis", 120, 78);
dogs.add(boi); dogs.add(clover); dogs.add(aiko);
dogs.add(zooey); dogs.add(charis);

List<Dog> heavyDogs =
    dogs.stream( )
        .filter(d -> d.getWeight( ) >= 50)
        .collect(Collectors.toList( ));
```

Entonces podemos imprimir la lista:

```
heavyDogs.forEach(System.out::println);
```

Recogiendo valores desde Streams

El método **collect()** es una operación de reducción: reduce un stream en una colección de objetos o un valor. El **Collector** que pasamos al método especifica cómo reducir el stream, por ejemplo, en una List o un Set. La clase **Collectors** proporciona implementaciones de **Collector** que determinan cómo recopilar, todo, desde métodos para hacer una lista simple, como **toList()**, hasta métodos que nos permiten agrupar elementos en una Collection, resultando en mapa de forma que los valores están organizados por el valor de la clave. Al recolectar podemos:

- Agrupar valores juntos basándonos en una **Function** (devuelve un **Map**).
- Partición de valores en particiones true/false basadas en un **Predicate** (devuelve un **Map**).
- Mapea valores a otros valores utilizando una **Function** (devuelve un **Collector**).
- Unir valores en una String.
- Contar valores a medida que recopilamos.

```
class Person {
    public String name;
    public Integer age;
    public Person(String name, Integer age) {
        this.name = name; this.age = age;
    }
    public String getName( ) { return this.name; }
    public String getAge( ) { return this.age; }
    public String toString( ) { return this.name + " tiene " + this.age + " años."; }
}
```

```
Person beth = new Person("Beth",30);
Person eric = new Person("Eric",31);
Person deb = new Person("Deb",31);
Person liz = new Person("Liz",30);
Person wendi = new Person("Wendi",34);
Person kathy = new Person("Kathy",35);
Person bert = new Person("Bert",32);
Person bill = new Person("Bill",34);
Person robert = new Person("Robert",38);
```

```
List<Person> people = new ArrayList<Person>( );
people.add(beth); people.add(eric); people.add(deb);
people.add(liz); people.add(wendi); people.add(kathy);
people.add(bert); people.add(bill); people.add(robert);
```

```
List<Person> peopleAge34 = people.stream( )
    .filter(p -> p.getAge( ) == 34)
    .collect(Collectors.toList( )); // Los recogemos en una nueva List
System.out.println("Gente de 34 : " peopleAge34);
```

No hay garantía de qué tipo de List obtenemos usando **Collectors.toList()**. Si deseamos específicamente una **ArrayList**, podemos usar el método **toCollection()** en su lugar, así:

```
List<Person> peopleAge34 = people.stream( )
    .filter(p -> p.getAge( ) == 34)
    .collect(Collectors.toCollection(ArrayList::new));
```

Observemos aquí que estamos pasando una referencia a método a **Collectors**. El método **toCollection()**. Mirando la documentación para **Collectors.toCollection()**, vemos que este método requiere un **Supplier** cuyo método funcional debe devolver una nueva **Collection** vacía del "tipo apropiado". Podríamos escribir el código así:

```
ArrayList<Dog> heavyDogs = dogs.stream( )
    .filter(d -> d.getWeight( ) >= 50)
    .collect(Collectors.toCollection( ( ) -> new ArrayList<Dog>( ) ));
```

proporcionando el **Supplier () -> new ArrayList<Dog>()** como argumento a **Collectors.toCollection()**. Recordemos que una referencia a método es una abreviatura

para una expresión lambda que simplemente rodea y llama a otro método. Entonces, escribiendo **ArrayList::new** en lugar de la lambda, estamos diciendo lo mismo que la expresión lambda anterior: es decir, simplemente llamamos al constructor (con **new**) de un **ArrayList<Dog>** para crear un nuevo ArrayList.

Usando collect() con Files.lines()

En este ejemplo anterior, transmitimos datos en forma de String sobre DVD desde el archivo y agregamos DVD de uno en uno a una Lista en un consumidor **forEach()**:

```
List<DVDInfo> dvds = new ArrayList<DVDInfo>( );
try ( Stream<String> stream = Files.lines(Paths.get(filename))) {
    stream.forEach( line -> {
        String[] dvdItems = line.split("/");
        DVDInfo dvd = new DVDInfo(dvdItems[0], dvdItems[1], dvdItems[2] );
        dvds.add(dvd);
    });
}
```

Ahora que sabemos cómo usar **collect()**, probablemente podamos pensar en una Manera mejor de hacer esto, ¿verdad? Si bien la solución anterior funciona, no es la forma recomendada de recopilar elementos de un stream en una List porque estamos modificando un objeto que se define fuera de la tubería de stream, en otras palabras, la tubería de flujo tiene un efecto secundario. El uso **de collect()** es la forma preferida

de hacer esto y, como veremos, junto con evitar un efecto secundario innecesario, también hace que nuestro código sea más claro.

```
String filename = "names.txt";
try ( Stream<String> stream = Files.lines(Paths.get(filename))) {
    List<String> data = stream.collect(Collectors.toList( ));
    data.forEach(System.out::println);
} catch(IOException e){
    System.out.println(e);
}
```

Agrupando y particionando

¿Y si queremos agrupar a las personas por edad? Podemos usar el método **Collectors.groupingBy()**. Especificamos cómo agrupar objetos **Person** pasando una **Function** al método **groupingBy()**, que devuelve un **Collector** que recopilará elementos de datos desde el stream y los agrupará en un **Map** por una clave, de acuerdo con esa **Function**.

```
Map<Integer,List<Person>> peopleByAge = people.stream( )
    .collect(Collectors.groupingBy(Person::getAge));
System.out.println("Gente por edad: " + peopleByAge);
```

Tengamos en cuenta que el **Map** que se obtiene utiliza una edad (**Integer**) como clave y el valor es una **List** del mismo tipo que el del objeto en el stream; en este caso, ese es **Person**. Observemos también que la forma en que **System.out.println()** nos muestra un **Person** en la salida es usando el método **toString()**, pero los valores en la **List** asociados con cada uno no son **Strings**; son **Person** (se puede ver desde el tipo parámetro en la **List<Person>** que ese es el caso).

El método **groupingBy()** está muy sobrecargado con una variedad de opciones para agrupar valores en una **Collection**. Por ejemplo, podemos usar una versión de **groupingBy()** que toma una función de clasificación como primer argumento y un **Collector** como segundo argumento. Ese argumento **Collector** nos permite reducir los valores del **Map** que obtenemos como resultado del método **groupingBy()**, reduciendo así cada **List** de valores en otro valor.

Aquí, vamos a agrupar personas por edad, pero en lugar de crear una **List** de las personas asociadas con cierta edad en un **Map**, como hicimos anteriormente, ahora estamos va a contar el número de personas en la lista asociadas con una edad determinada y usaremos ese valor en el **Map** resultante en su lugar. Entonces tenemos dos reducciones: tenemos una reducción **groupingBy()** para agrupar personas por edad y tenemos una reducción **counting()** para contar las personas en la **List** asociadas con una edad particular:

```
Map<Integer, Long> numPeopleWithAge = people.stream( )
    .collect(Collectors.groupingBy(Person::getAge,Collectors.counting( )));
System.out.println("Gente por edad: " + numPeopleWithAge);
```

¿Qué pasa si queremos agrupar a las personas por edad, pero listar solo su nombre en vez de todo el objeto **Person** en el **Map**? Es decir, queremos un mapa de edades y nombres (**Map<Integer, List<String>>**) en lugar de un mapa de edades y objetos **Person** (**Map<Integer, List<Person>>**)? Podemos hacerlo pasando un Collector **Collectors.mapping()** como ese segundo argumento para **groupingBy()**:

```
Map<Integer, List<String>> namesByAge = people.stream( )
    .collect( Collectors.groupingBy( Person::getAge,
    Collectors.mapping( Person::getName, Collectors.toList( ) ) ) );
System.out.println(" Gente por edad: " + namesByAge);
```

El método **mapping()** asigna cada persona a otro valor. Lo que nos gustaría hacer es mapear cada **Person** a su nombre. Echando un vistazo al método **mapping()**, vemos que su primer argumento es una **Function** cuyo método funcional toma un objeto del tipo del que estamos mapeando (una **Persona**) y devuelve otro objeto, en este caso, el nombre del **Person**, una **String**. El segundo argumento de **mapping()** es un recopilador que nos dice qué hacer con los potencialmente múltiples valores que estamos mapeando.

Recordemos que podemos tener varias personas de la misma edad, por lo que estamos asignando una **List** de objetos **Person** a una **List** de sus nombres. Ese segundo argumento especifica cómo nos gustaría recopilar esos nombres Usaremos **toList()** para mantenerlo fácil, pero también puede elegir **toCollection()** para crear una **ArrayList** o alguna otra colección para los nombres.

Particionar a medida que recopilamos es esencialmente un tipo más especializado de **groupingBy()**. El método **partitioningBy()** organiza los resultados en un **Map** como hace **groupingBy()**, pero **partitioningBy()** toma un **Predicate** en lugar de una **Function**, así los resultados se dividen en dos grupos (particiones) en función de si los elementos en el stream pasan la prueba en el **Predicate**.

```
Map<Boolean, List<Person>> peopleOlderThan34 = people.stream( )
    .collect(
    Collectors.partitioningBy( p -> p.getAge( ) > 34 ) );
System.out.println("Gente > 34: " + peopleOlderThan34);
```

Hemos deslizado otro ejemplo de una referencia a un método de instancia: **Persona :: getAge**. Hicimos esto para agrupar personas por edad. **groupingBy()** toma una función que mapea un objeto **Person** a otro valor para usar ese valor como clave de mapeo; en este caso, diciendo a **groupingBy()** para mapear personas por edad.

La referencia a método **Person :: getAge** es una referencia a método de instancia que se refiere a un método no estático de una instancia de **Person**. Si escribiéramos esa referencia de método como una expresión lambda, sería así:


```
(p) -> p.getAge( )
```

Entonces, esta lambda es una **Function** cuyo método funcional toma un objeto de tipo `Person` y devuelve un objeto de tipo `Integer`, asignando un objeto `Person` a su edad.

Sumando y promediando

Un par de otros colectores útiles son **summingInt()** y **averagingInt()** (y sus contrapartes `long` y `double`). Como podemos suponer, estos necesitan trabajar en números, así que ambos toman una función **ToIntFunction** (que es una función que asigna un objeto a un `int`). Añadimos nuevos elementos a la lista:

```
Person bill2 = new Person("Bill", 40);
Person beth2 = new Person("Beth", 45);
Person bert2 = new Person("Bert", 38);
people.add(bill2); people.add(beth2); people.add(bert2);
```

Primero, queremos sumar las edades de los dos Berts, los dos Beths y los dos Bills y agrupar por nombre:

```
Map<String, Integer> sumOfBAges = people.stream( )
    .filter(p -> p.getName( ).startsWith("B"))
    .collect(
        Collectors.groupingBy(
            Person::getName,
            Collectors.summingInt(Person::getAge)
        )
    );
System.out.println("Gente cuyo nombre empieza por B por suma de edad: " +
    sumOfBAges);
```

¿Y la media de edad de la gente cuyo nombre empieza por "B"?

```
Map<String, Integer> avgOfBAges = people.stream( )
    .filter(p -> p.getName( ).startsWith("B"))
    .collect(
        Collectors.groupingBy(
            Person::getName,
            Collectors.averagingInt(Person::getAge)
        )
    );
System.out.println("Gente cuyo nombre empieza por B por media de edad: " +
    avgOfBAges);
```

Es importante tener en cuenta aquí que el **Collector averagingInt()** reduce las edades a un `Double`, por lo que el tipo de la signatura en este `Map` es **Map<String,Double>**

(comparar con el tipo de la signatura en el resultado del **Collector summingInt()**, que es **Map<String,Integer>**).

Contando, uniendo, maxBy y minB

Hemos utilizado el método **Collectors.counting()** anterior para crear un **Collector** que cuenta los elementos que se recopilan. Un simple ejemplo de uso **counting()** es:

```
Long n = people.stream( ).collect(Collectors.counting( ));
System.out.println("Cuenta: " +n);
```

El método **Collectors.joining()** devuelve un recopilador que toma elementos del stream y los concatena en una cadena por el orden en el que se han encontrado (que puede o no ser el orden de la Colección original estás transmitiendo!).

```
String older34 = people.stream( )
    .filter(p -> p.getAge( ) > 34)
    .map(Person::getName)
    .collect(Collectors.joining(" , "));
System.out.println(" Nombres de gente mayor de 34: " + older34);
```

El método **maxBy()** toma un **Comparator** (al igual que **minBy()**) para comparar los elementos de un stream a medida que se recopilan y devuelve un **Collector** que, cuando se usa con el método **Stream.collect()**, reducirá el stream al "max" de los elementos en el stream medido por ese Comparador.

Hay que tener en cuenta que **maxBy()** devuelve un **Optional<Person>**, no un **Person**, porque al encontrar el máximo mientras recopila un stream, si ese stream está vacío, no habrá valor. Entonces usamos el método **ifPresent()** del **Optional** para asegurarnos de que haya un valor allí antes de intentar imprimirlo.

```
people.stream( ).mapToInt (p -> p.getAge( )).max( );
```

y nuestro resultado es un objeto Person. Recordemos, **IntStream.max()** funciona solo en números, así que cuando mapeamos nuestro stream de objetos Person a un stream de **Integers** para usar **max()** para encontrar a la persona más vieja, obtuvimos un número de vuelta. Al usar **maxBy()** con **collect()**, podemos encontrar el objeto Person que tiene la edad más alta y recupera a todo el objeto Person, no solo su edad.

Métodos Stream para coleccionar y sus Collectors

Interface	Method	Description
Stream	<code>collect(Collector<? super T, A, R> collector)</code>	Reduces a stream to an instance of a mutable type, like a Collection. The Collector stands in for a constructor, an accumulator, and a combiner (see below).
Stream	<code>collect(Supplier<R> supplier, BiConsumer<R, ? super T> accumulator, BiConsumer<R, R> combiner)</code>	Reduces a stream to an instance of a mutable type. Supplier provides a new instance of the mutable type such as a Collection; accumulator adds each element of the stream to the collection; combiner specifies how to merge elements from one collection to another. A Collector can be used to capture all three components.
DoubleStream, IntStream, LongStream	<code>collect(Supplier<R> supplier, ObjDoubleConsumer<R> accumulator, BiConsumer<R, R> combiner)</code> <code>collect(Supplier<R> supplier, ObjIntConsumer<R> accumulator, BiConsumer<R, R> combiner)</code> <code>collect(Supplier<R> supplier, ObjLongConsumer<R> accumulator, BiConsumer<R, R> combiner)</code>	Reduces a stream to an instance of a mutable type.

Method	Description
<code>toList()</code>	Returns a <code>Collector</code> that accumulates elements into a new <code>List</code> .
<code>toMap(Function<? super T, ? extends K> keyMapper, Function<? super T, ? extends U> valueMapper)</code>	Returns a <code>Collector</code> that accumulates elements into a new <code>Map</code> .
<code>toSet()</code>	Returns a <code>Collector</code> that accumulates elements into a new <code>Set</code> .
<code>toCollection(Supplier<C> collectionFactory)</code>	Returns a <code>Collector</code> that accumulates elements into a new <code>Collection</code> .
<code>groupingBy(Function<? super T, ? extends K> classifier)</code>	Returns a <code>Collector</code> that implements the group by operation, which groups elements by a classification <code>Function</code> , and returns the results in a <code>Map</code> . We used this to map <code>Person</code> objects by age.
<code>groupingBy(Function<? super T, ? extends K> classifier, Collector<? super T, A, D> downstream)</code>	Returns a <code>Collector</code> that groups elements according to a classification <code>Function</code> and then performs an additional reduction on the values associated with a given key in the resulting <code>Map</code> . We used this to count the number of people of a certain age and map the count to age.
<code>partitioningBy(Predicate<? super T> predicate)</code>	Returns a <code>Collector</code> that partitions elements according to the <code>Predicate</code> (into true and false partitions) and returns a <code>Map</code> . This is a more specific version of <code>groupingBy</code> .
<code>mapping(Function<? super T, ? extends U> mapper, Collector<? super U, A, R> downstream)</code>	Modifies a <code>Collector</code> that accepts elements of type <code>U</code> into a <code>Collector</code> that accepts elements of type <code>T</code> by applying the mapper <code>Function</code> to each element before it's accumulated in the <code>Collector</code> 's reduction. We used this to map a <code>Person</code> object to a <code>String</code> (the person's name) before grouping by the person's age.

Method	Description
<code>summingInt(ToIntFunction<? super T> mapper)</code>	Returns a Collector that computes the sum of input elements. We used this to sum the ages (type Integer) of Person objects with the same name; we used the mapper to map a Person to their age. Similar methods are <code>summingDouble</code> and <code>summingLong</code> (producing a Double sum and a Long sum, respectively). If no elements are present, the result is 0.
<code>averagingInt(ToIntFunction<? super T> mapper)</code>	Returns a Collector that computes the average of input elements. We used this to create a Map of Person names and their average age (type Double). Similar methods are <code>averagingDouble</code> and <code>averagingLong</code> (both of which produce Double values for the average). The mapper we used maps a Person to their age.
<code>counting()</code>	Returns a Collector that counts the number of elements and returns a long. If no elements are present, the result is 0.
<code>joining()</code>	Returns a Collector that concatenates elements into a String. We used this to concatenate Person names to associate a String of names with an age.
<code>maxBy(Comparator<? super T> comparator)</code>	Returns a Collector that returns the maximum element determined by the Comparator.
<code>minBy(Comparator<? super T> comparator)</code>	Returns a Collector that returns the minimum element determined by the Comparator.

Streams de Streams

Imaginemos que tenemos un archivo que contiene palabras separadas por espacios en múltiples líneas. Deseamos leer el archivo y determinar cuántas veces la palabra "Java" aparece en el archivo:

```
Stream<String> input = Files.lines(Paths.get("java.txt"));
```

Ahora, usted sabe que el stream de entrada va a transmitir una sola línea a la vez desde el archivo "java.txt", y para obtener palabras, en lugar de líneas, podemos dividir cada línea, así:

```
input.map(line -> line.split(" "))
```

El método **split()** en String crea un array de cadenas, y sabemos que **map()** produce un stream, así que esta línea de código toma el stream de líneas que provienen del archivo, se divide cada línea en un **String[]**, y se genera un Stream de arrays de String, que escribimos como **Stream<String[]>**:

```
Stream<String[ ]> inputStream = input.map(line -> line.split(" "));
```

Lo que queremos son las cadenas individuales en cada array, para que podamos filtrar las palabras que son iguales a "Java" y contarlas para ver cuántas veces aparece "Java" en el archivo. ¿Cómo convertimos el **Stream<String[]>** en un stream de Strings?

¿Qué pasa si transmitimos cada una de las matrices en el stream? Sabemos que podemos crear un stream desde un array con **Arrays.stream()** y ese **inputStream** es un stream de arrays de Strings. ¿Qué tal mapear cada array a un stream, así cada array se convierte en un stream de strings, y luego lo procesamos?

```
inputStream.map(array -> Arrays.stream(array)).forEach(System.out::println);
```

No sale lo que esperamos. En realidad, lo que estamos creando con:

```
inputStream.map(array -> Arrays.stream(array))
```

es un stream de streams:

```
Stream<Stream<String>> ss = inputStream.map(array -> Arrays.stream(array));
```

Tratamos de imprimir esto:

```
ss.forEach(System.out::println);
```

Cada elemento que vemos mostrado es una **Stream<String>**, en lugar de una **String**. Cuando transmitimos cada array de Strings creados por la división, cada uno de esos streams se convierte en un elemento en el stream principal que está transmitiendo los arrays. Por lo tanto, un stream de streams.

El método **flatMap()** es similar a **map()** en que mapea un stream de un tipo en un stream de otro tipo, pero hace algo extra; **flatMap()** "Aplana" los streams, esencialmente las concatena en un solo stream. Eso reemplaza cada stream con sus contenidos, creando un stream de muchos.

```
Stream<String> ss = inputStream.flatMap(array -> Arrays.stream(array));  
ss.forEach(System.out::println);
```

Nuestro objetivo es contar el número de palabras igual a "Java". Esperemos que esa tarea sea más fácil ahora; solo necesitas **filter()** y **count()**.

```
try {
    long n = Files.lines(Paths.get("java.txt"))
        .map(line -> line.split(" "))
        .flatMap(array -> Arrays.stream(array))
        .filter(w -> w.equals("Java"))
        .count( );
    System.out.println("Número de veces que 'Java' aparece: " + n);
} catch(IOException e){
    System.out.println("Error: " + getMessage( ));
}
```

La interfaz **Stream** también incluye **flatMapToDouble()**, **flatMapToInt()** y **flatMapToLong()** para "aplanar" a un **DoubleStream**, un **IntStream** y un **LongStream**, respectivamente, en caso de que lo necesitemos. Cada una de los streams primitivos que acabamos de mencionar también incluyen sus propios métodos **flatMap()** que toman un **DoubleFunction**, **IntFunction** y **LongFunction**, respectivamente.

Generando Streams

La operación **iterate()** crea un Stream secuencial infinito, comenzando con el primer argumento (conocido como "semilla") seguido de elementos creados por el **UnaryOperator** que proporcionamos como segundo argumento. Nunca deberíamos hacer algo como:

```
Stream
    .iterate(0, s -> s + 1)
    .forEach(System.out::println);
```

Aquí, la operación terminal **forEach()** nunca terminará porque el stream es infinito. Debido a que **iterate()** crea una secuencia infinita, necesitamos alguna forma de limitar cuánto obtenemos del stream para no meternos en algún problema. Podemos hacer eso con la operación **limit()**:

```
Stream
    .iterate(0, s -> s + 1)
    .limit(4)
    .forEach(System.out::println);
```

Para trabajar de forma segura con streams infinitos, necesita una operación cortocircuito operación. Eso podría ser **limit()**, como usamos anteriormente, o también puede ser un operación como **findFirst()**, **findAny()** o **anyMatch()**.

Simulemos un sensor que genera un stream infinito de datos:

```
class Sensor {
    String value = "up";
    int i = 0;
    public Sensor( ) { }
    public String next( ) {
        i = i+1;
        return i > 10 ? "down" : "up";
    }
}
```

Ahora usemos este sensor con un flujo infinito. Para este ejemplo, vamos a usar el método **Stream.generate()**, que toma un **Supplier**:

```
Sensor s = new Sensor( );
Stream<String> sensorStream = Stream.generate( ( ) -> s.next( ));
```

Ahora podemos escribir código para buscar el primer valor "down" en el stream infinito así:

```
Optional<String> result = sensorStream
    .filter(v -> v.equals("down"))
    .findFirst( );
result.ifPresent(System.out::println);
```

El método **findFirst()** es un método de cortocircuito, por lo que tan pronto como encuentra un valor "down" en la secuencia, todo se detiene. En caso de no hubiese ningún "down", el stream seguiría generando valores y **findFirst()** nunca se completará.

Aunque **Stream.iterate()** es una buena manera de generar números hasta un cierto punto, otra forma de generar números es con **range()**. En la práctica, este en realidad podría ser más útil. Este es un método en los streams primitivos **IntStream** y **LongStream**:

```
IntStream numStream = IntStream.range(10,15);
numStream.forEach(System.out::println);
```

Si deseamos una secuencia de números que incluya el segundo argumento, usaremos **rangeClosed()**:

```
IntStream numStream = IntStream.rangeClosed(10,15);
numStream.forEach(System.out::println);
```

Podemos limitar el número de elementos a los cinco primeros:

```
IntStream evenBefore10 = IntStream
    .rangeClosed(0,20)
    .filter(i -> i % 2 == 0)
    .limit(5);
evenBefore10.forEach(System.out::println);
```

Y si en vez de esto, queremos evitar los primeros cinco items e imprimir solo números pares entre 10 y 20:

```
IntStream evensAfter10 = IntStream
    .rangeClosed(0,20)
    .filter(i -> i % 2 == 0)
    .skip(5);
evensAfter10.forEach(System.out::println);
```


Métodos para generar Streams

Method	Description
<code>iterate(T seed, UnaryOperator<T> function)</code>	Applies a function to a seed and then to each subsequent value, and returns an infinite stream consisting of the resulting elements. For example, we created an infinite stream by starting with a seed of 0 and using a function that adds 1 to its input, resulting in a stream of Integers: 0, 1, 2, 3... Primitive streams <code>DoubleStream</code>, <code>IntStream</code>, and <code>LongStream</code> each have their own versions that use the primitive unary operators for the function (<code>DoubleUnaryOperator</code>, <code>IntUnaryOperator</code>, and <code>LongUnaryOperator</code>) and whose seeds are of type <code>double</code>, <code>int</code>, and <code>long</code>, respectively.
<code>generate(Supplier<T> s)</code>	Returns an infinite stream consisting of elements generated by the provided <code>Supplier</code>. We used this to generate a stream of "up" and "down" values. Primitive streams <code>DoubleStream</code>, <code>IntStream</code>, and <code>LongStream</code> have their own versions that use the primitive suppliers (<code>DoubleSupplier</code>, <code>IntSupplier</code>, and <code>LongSupplier</code>) and generate primitive streams, respectively.
<code>range(int startInclusive, int endExclusive),</code> <code>range(long startInclusive, long endExclusive)</code>	Primitive streams <code>IntStream</code> and <code>LongStream</code> only: this method produces a sequential stream of elements, incrementing by 1 (e.g., 0, 1, 2, 3...), not including the <code>endExclusive</code> value.
<code>rangeClosed(int startInclusive, int endInclusive),</code> <code>rangeClosed(long startInclusive, long endInclusive)</code>	Primitive streams <code>IntStream</code> and <code>LongStream</code> only: this method produces a sequential stream of elements, incrementing by 1 (e.g., 0, 1, 2, 3...), including the <code>endInclusive</code> value.
<code>limit(long maxSize)</code>	Returns a stream consisting of the elements of the stream on which it's called, limited to <code>maxSize</code> number of elements (for all stream types).
<code>skip(long n)</code>	Returns a stream consisting of the elements of the stream on which it's called, after <code>n</code> elements from the stream have been skipped (for all stream types).

Una introducción a los Streams paralelos

Hasta ahora, todas las transmisiones con las que hemos trabajado tienen sido streams en serie: streams que procesan un elemento de datos a la vez. Los **streams paralelos** pueden procesar elementos en un stream simultáneamente.

La forma en que Java lo hace esto es dividir el stream en substreams y luego ejecutar las operaciones definidas en la tubería de stream en cada uno de estos substreams simultáneamente, lo que significa que cada substream se procesa en un hilo.

Supongamos que tenemos algunos números y deseamos sumarlos:

```
List<Integer> nums = Arrays.asList( 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 );
int sum = nums.stream( )
    .mapToInt(n -> n)
    .sum( );
System.out.println("La suma es: " + sum);
```

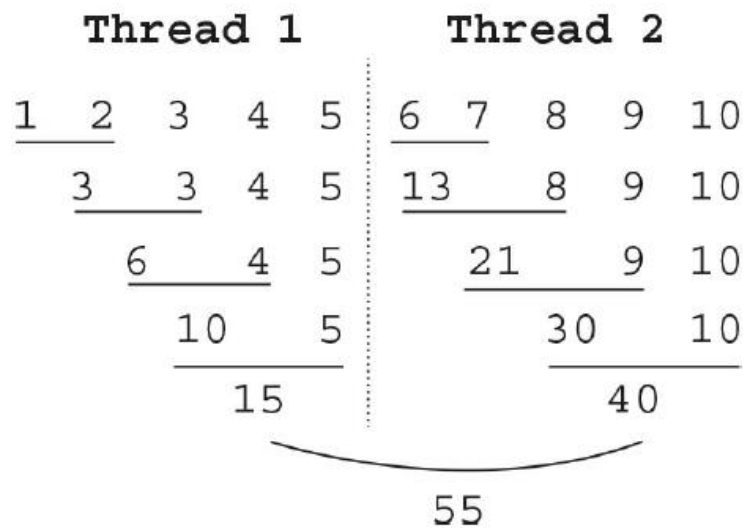
Este stream es un stream en serie, por lo que cada número se agrega a la suma uno cada vez. Para hacer de esto un stream paralelo, simplemente llamamos al método **parallel()** en la stream antes de sumar:

```
List<Integer> nums = Arrays.asList( 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 );
int sum = nums.stream( )
    .parallel( )
    .mapToInt(n -> n)
    .sum( );
System.out.println("La suma es: " + sum);
```

Para una transmisión en serie, tenemos un hilo que maneja toda la operación de suma:

[illegible]

Para un stream paralelo, tenemos múltiples hilos que manejan la operación de suma concurrentemente:



Una cosa a tener en cuenta con streams paralelos es realizar una operación que se basa en un **orden específico**. Si usamos un stream paralelo, podemos obtener resultados inesperados. Comparemos el uso de **forEach()** para mostrar los elementos en un stream cuando el stream es serial y cuando la transmisión es paralela.

```
Arrays.asList("boi","charis","zooey","aiko")
.stream( )
.forEach(System.out::println);
```

Si lo paralelizamos, el orden de impresión de los elementos puede variar:

```
Arrays.asList("boi","charis","zooey","aiko")
.stream( )
.parallel( )
.forEach(System.out::println);
```